Python & Go

Comprehensive Language Guide

From Fundamentals to Advanced Concepts
For Infrastructure and Database Automation Engineers

Covering Language Internals, Concurrency, Networking, Database Access, Kubernetes Integration, CLI Tooling, Testing, and Production Patterns

April 2026

Table of Contents

Part I

Python: Fundamentals to Advanced

Python is an interpreted, dynamically typed, high-level language designed for readability and rapid development. Created by Guido van Rossum in 1991, it has become the dominant language for automation, scripting, data engineering, and infrastructure tooling across the technology industry.

This part covers Python from first principles through advanced production patterns, with every example grounded in the infrastructure and database administration context.

1. Language Overview and Philosophy

Python follows a philosophy captured in its Zen (PEP 20): readability counts, explicit is better than implicit, and there should be one obvious way to do it. These principles make Python code approachable and maintainable across teams.

1.1 How Python Executes Code

When you run a Python script, the CPython interpreter performs two phases. First, it compiles your source code into bytecode — an intermediate representation stored in .pyc files under the __pycache__ directory. Second, the Python Virtual Machine (PVM) executes this bytecode instruction by instruction.

This is fundamentally different from compiled languages like Go. There is no separate compile step that produces a native binary. The interpreter must be present on the target machine to run Python code, which is why virtual environments and dependency management matter so much in production.

1.2 Dynamic Typing

Variables in Python have no declared type. A name simply binds to an object at runtime:

```
x = 42          # x refers to an int object
x = "hello"     # now x refers to a str object — no error
x = [1, 2, 3]   # now a list — still no error
```

The type lives with the object, not the variable. This gives you flexibility and speed during development, but it also means type errors surface only at runtime. Modern Python mitigates this with type hints (covered in Chapter 15) and static analysis tools like mypy.

1.3 Memory Management

Python uses automatic memory management combining reference counting with a cyclic garbage collector. Every object tracks how many names refer to it. When that count drops to zero, the memory is freed immediately. For objects that reference each other in cycles, the garbage collector periodically detects and cleans them up.

The Global Interpreter Lock (GIL) is a mutex that ensures only one thread executes Python bytecode at a time within a single process. For CPU-bound tasks, this limits true parallelism. For I/O-bound tasks — database queries, API calls, file operations — the GIL is released during blocking operations, so threading and asyncio remain effective.

> *Practical impact: For DBA automation scripts that call PostgreSQL or MongoDB, the GIL is rarely a bottleneck. Your scripts spend most of their time waiting for database responses, not crunching numbers.*

2. Environment Setup and Tooling

2.1 Installation on AlmaLinux

AlmaLinux 8 ships with Python 3.6 as the system default, and AlmaLinux 9 ships with Python 3.9. For modern tooling, install a newer version alongside the system Python without replacing it:

```
# AlmaLinux 9 — install Python 3.11 from AppStream
sudo dnf install python3.11 python3.11-pip python3.11-devel

# Verify
python3.11 --version
```

> *Never replace the system Python on RHEL-family distros. System tools like dnf depend on it. Always install additional versions side by side.*

2.2 Virtual Environments

Virtual environments isolate project dependencies from the system Python and from each other. This is critical on servers where you may have multiple automation tools with conflicting dependency versions.

```
# Create a virtual environment
python3.11 -m venv /apps/tools/pg-monitor/venv

# Activate it
source /apps/tools/pg-monitor/venv/bin/activate

# Install dependencies
pip install psycopg2-binary pymongo fastapi uvicorn

# Freeze for reproducibility
pip freeze > requirements.txt

# Deactivate when done
deactivate
```

2.3 Key Tools

The Python ecosystem offers mature tooling for every stage of development:

- pip — Package installer. Use with requirements.txt or constraints files for reproducible installs.
- venv / virtualenv — Dependency isolation. venv is built into Python 3.3+.
- poetry — Dependency management and packaging with lock files, similar to Go modules in spirit.
- black — Opinionated code formatter. Eliminates style debates.
- ruff — Extremely fast linter written in Rust. Replaces flake8, isort, and several others.
- mypy — Static type checker. Catches type errors before runtime when you use type hints.
- pytest — Testing framework. More powerful and readable than unittest.

3. Core Syntax and Data Types

3.1 Variables and Basic Types

Python's core types include integers (arbitrary precision), floats (64-bit IEEE 754), strings (Unicode by default), booleans, and None. Variables are created by assignment — no declaration keyword needed.

```
# Integers — arbitrary precision
max_connections = 200
wal_size_bytes = 16 * 1024 * 1024 * 1024   # 16 GB, no overflow

# Floats
replication_lag_seconds = 0.045

# Strings — single, double, or triple quotes
pg_data = "/apps/pgsql_data/16"
query = """
    SELECT pg_size_pretty(pg_database_size('production'))
    AS db_size;
"""

# Boolean
is_primary = True

# None — represents absence of a value
standby_slot = None
```

3.2 Collections

Python provides four essential built-in collection types, each with distinct characteristics:

Lists — Ordered, Mutable Sequences

```
replicas = ["standby1", "standby2", "standby3"]
replicas.append("standby4")
replicas.remove("standby2")

# List comprehension
healthy = [r for r in replicas if check_health(r)]

# Slicing
first_two = replicas[:2]
last_one = replicas[-1]
```

Dictionaries — Key-Value Mappings

```
pg_config = {
    "shared_buffers": "8GB",
    "work_mem": "256MB",
    "max_connections": 200,
    "wal_level": "replica",
```

```
}

# Access and update
pg_config["effective_cache_size"] = "24GB"
work_mem = pg_config.get("work_mem", "64MB")  # safe access with default

# Dictionary comprehension
sizes = {db: get_size(db) for db in databases}
```

Tuples — Ordered, Immutable Sequences

```
# Tuples cannot be modified after creation
server = ("pgprimary01", 5432, "production")
host, port, env = server   # tuple unpacking

# Common use: returning multiple values
def get_replication_status():
    return ("streaming", 0.003, True)

state, lag, is_sync = get_replication_status()
```

Sets — Unordered, Unique Collections

```
active_dbs = {"production", "staging", "analytics"}
backup_dbs = {"production", "analytics", "archive"}

# Set operations
needs_backup = active_dbs - backup_dbs      # {"staging"}
all_dbs = active_dbs | backup_dbs           # union
common = active_dbs & backup_dbs            # intersection
```

4. Control Flow and Functions

4.1 Conditionals

Python uses if/elif/else with indentation-based blocks. There are no braces or end keywords.

```
lag = get_replication_lag()

if lag > 30:
    alert("CRITICAL: replication lag exceeds 30 seconds")
    initiate_failover_check()
elif lag > 5:
    alert("WARNING: replication lag is elevated")
else:
    log("Replication lag normal: {:.3f}s".format(lag))
```

4.2 Loops

```
# For loop — iterates over any iterable
databases = ["production", "staging", "analytics"]
for db in databases:
    size = get_db_size(db)
    print(f"{db}: {size}")

# For with index
for i, db in enumerate(databases):
    print(f"{i+1}. {db}")

# While loop
retry_count = 0
while retry_count < 3:
    if try_connect():
        break
    retry_count += 1
    time.sleep(2 ** retry_count)  # exponential backoff
```

4.3 Functions

Functions are defined with the def keyword. Python supports default arguments, keyword arguments, and variable-length argument lists.

```
def run_pg_query(host, port=5432, database="production",
                 timeout=30, readonly=True):
    """Execute a query against a PostgreSQL instance."""
    dsn = f"host={host} port={port} dbname={database}"
    conn = psycopg2.connect(dsn, connect_timeout=timeout)
    if readonly:
        conn.set_session(readonly=True)
    return conn

# Calling with keyword arguments
```

```
conn = run_pg_query("pgprimary01", database="analytics",
                    timeout=60)

# *args and **kwargs
def log_event(*args, **kwargs):
    timestamp = kwargs.get("timestamp", datetime.now())
    severity = kwargs.get("severity", "INFO")
    message = " ".join(str(a) for a in args)
    print(f"[{timestamp}] [{severity}] {message}")
```

4.4 Lambda and Higher-Order Functions

```
# Lambda — anonymous single-expression function
servers = [("pg01", 0.5), ("pg02", 12.3), ("pg03", 0.1)]
sorted_by_lag = sorted(servers, key=lambda s: s[1])

# map, filter, reduce
sizes = list(map(get_db_size, databases))
large_dbs = list(filter(lambda s: s > 100_000_000, sizes))
```

5. Object-Oriented Programming

5.1 Classes and Objects

```
class DatabaseConnection:
    """Manages a connection to a PostgreSQL instance."""

    def __init__(self, host, port=5432, database="postgres"):
        self.host = host
        self.port = port
        self.database = database
        self._conn = None

    def connect(self):
        import psycopg2
        dsn = f"host={self.host} port={self.port} dbname={self.database}"
        self._conn = psycopg2.connect(dsn)
        return self

    def execute(self, query, params=None):
        with self._conn.cursor() as cur:
            cur.execute(query, params)
            return cur.fetchall()

    def close(self):
        if self._conn:
            self._conn.close()
            self._conn = None

    def __enter__(self):
        return self.connect()

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.close()
        return False
```

5.2 Inheritance

```
class MongoConnection(DatabaseConnection):
    """Extends base connection for MongoDB."""

    def __init__(self, host, port=27017, database="admin"):
        super().__init__(host, port, database)

    def connect(self):
        from pymongo import MongoClient
        self._conn = MongoClient(self.host, self.port)
        return self

    def execute(self, collection, query):
        db = self._conn[self.database]
```

```
        return list(db[collection].find(query))
```

5.3 Properties and Encapsulation

```
class PGCluster:
    def __init__(self, primary_host, standbys):
        self._primary = primary_host
        self._standbys = list(standbys)

    @property
    def primary(self):
        return self._primary

    @primary.setter
    def primary(self, new_primary):
        if new_primary not in self._standbys:
            raise ValueError(f"{new_primary} is not a known standby")
        self._standbys.remove(new_primary)
        self._standbys.append(self._primary)
        self._primary = new_primary
```

5.4 Abstract Base Classes

```
from abc import ABC, abstractmethod

class HealthCheck(ABC):
    @abstractmethod
    def check(self) -> bool:
        """Return True if the service is healthy."""
        pass

    @abstractmethod
    def name(self) -> str:
        pass

class PGHealthCheck(HealthCheck):
    def check(self) -> bool:
        return run_query("SELECT 1") == [(1,)]

    def name(self) -> str:
        return "PostgreSQL Primary"
```

6. Modules, Packages, and Virtual Environments

6.1 Module System

Every .py file is a module. Directories containing an __init__.py file are packages. Python searches for modules in sys.path, which includes the current directory, installed packages, and the standard library.

```
# Importing
import os
import json
from pathlib import Path
from datetime import datetime, timedelta

# Relative imports within a package
# dba_toolkit/
#   __init__.py
#   pg/
#     __init__.py
#     health.py
#     backup.py
#   mongo/
#     __init__.py
#     health.py

# Inside dba_toolkit/pg/backup.py:
from .health import check_primary_status
from ..mongo.health import check_replica_set
```

6.2 Creating Distributable Packages

For tools that need to be installed across multiple servers, structure your project with a pyproject.toml (modern standard) or setup.py:

```
# pyproject.toml
[project]
name = "dba-toolkit"
version = "1.0.0"
requires-python = ">=3.9"
dependencies = [
    "psycopg2-binary>=2.9",
    "pymongo>=4.0",
    "click>=8.0",
]

[project.scripts]
pg-health = "dba_toolkit.pg.health:main"
mongo-check = "dba_toolkit.mongo.health:main"
```

7. File I/O and Data Serialization

7.1 Reading and Writing Files

```
from pathlib import Path

# Reading a config file
config_path = Path("/apps/pgsql_data/16/postgresql.conf")
content = config_path.read_text()

# Writing output
output_path = Path("/apps/logs/health_check.log")
output_path.write_text(f"Check completed at {datetime.now()}\n")

# Context manager for large files
with open("/apps/logs/postgresql.log") as f:
    for line in f:      # streams line by line — memory efficient
        if "ERROR" in line:
            process_error(line)
```

7.2 JSON and YAML

```
import json
import yaml   # pip install pyyaml

# JSON — standard for API responses and configs
config = {"max_connections": 200, "wal_level": "replica"}
json_str = json.dumps(config, indent=2)
parsed = json.loads(json_str)

# YAML — common in Kubernetes manifests and Ansible
with open("pg-cluster.yaml") as f:
    manifest = yaml.safe_load(f)

manifest["spec"]["replicas"] = 3
with open("pg-cluster-updated.yaml", "w") as f:
    yaml.dump(manifest, f, default_flow_style=False)
```

7.3 CSV and Structured Data

```
import csv

# Writing query results to CSV
with open("/apps/reports/db_sizes.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["database", "size_mb", "tables"])
    for db in databases:
        writer.writerow([db.name, db.size_mb, db.table_count])

# Reading CSV
```

```
with open("/apps/reports/db_sizes.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"{row['database']}: {row['size_mb']} MB")
```

8. Error Handling and Logging

8.1 Exceptions

Python uses a try/except/else/finally pattern. Exceptions propagate up the call stack until caught. This differs fundamentally from Go's explicit error return values.

```
import psycopg2
import logging

logger = logging.getLogger(__name__)

def safe_query(dsn, query, params=None):
    conn = None
    try:
        conn = psycopg2.connect(dsn, connect_timeout=10)
        with conn.cursor() as cur:
            cur.execute(query, params)
            results = cur.fetchall()
    except psycopg2.OperationalError as e:
        logger.error("Connection failed: %s", e)
        raise
    except psycopg2.ProgrammingError as e:
        logger.error("Query error: %s", e)
        return None
    else:
        logger.info("Query returned %d rows", len(results))
        return results
    finally:
        if conn:
            conn.close()
```

8.2 Custom Exceptions

```
class DBAToolkitError(Exception):
    """Base exception for all DBA toolkit errors."""
    pass

class ReplicationLagError(DBAToolkitError):
    def __init__(self, lag_seconds, threshold):
        self.lag = lag_seconds
        self.threshold = threshold
        super().__init__(
            f"Replication lag {lag_seconds:.3f}s exceeds "
            f"threshold {threshold}s"
        )
```

8.3 Structured Logging

```
import logging
```

```
import json

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_entry = {
            "timestamp": self.formatTime(record),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }
        if record.exc_info:
            log_entry["exception"] = self.formatException(
                record.exc_info
            )
        return json.dumps(log_entry)

# Setup
handler = logging.FileHandler("/apps/logs/dba-toolkit.log")
handler.setFormatter(JSONFormatter())
logger = logging.getLogger("dba_toolkit")
logger.addHandler(handler)
logger.setLevel(logging.INFO)
```

9. Concurrency and Parallelism

9.1 Threading — I/O-Bound Concurrency

Threading is effective for I/O-bound operations where your code spends most of its time waiting for external responses (database queries, HTTP calls, file reads). The GIL is released during these blocking operations.

```
from concurrent.futures import ThreadPoolExecutor
import psycopg2

servers = ["pg01", "pg02", "pg03", "pg04", "pg05"]

def check_health(host):
    try:
        conn = psycopg2.connect(
            f"host={host} dbname=postgres", connect_timeout=5
        )
        conn.close()
        return (host, "healthy")
    except Exception as e:
        return (host, f"error: {e}")

# Check all servers concurrently
with ThreadPoolExecutor(max_workers=10) as executor:
    results = list(executor.map(check_health, servers))

for host, status in results:
    print(f"{host}: {status}")
```

9.2 Asyncio — Event-Loop Concurrency

Asyncio uses cooperative multitasking with a single thread. Functions declare themselves as async and use await to yield control during I/O. This is more efficient than threading for high-concurrency network operations.

```
import asyncio
import asyncpg

async def get_db_size(host, database):
    conn = await asyncpg.connect(
        host=host, database=database, timeout=10
    )
    row = await conn.fetchrow(
        "SELECT pg_database_size($1) AS size", database
    )
    await conn.close()
    return (host, database, row["size"])

async def check_all_instances():
    tasks = [
        get_db_size("pg01", "production"),
        get_db_size("pg02", "analytics"),
```

```
            get_db_size("pg03", "staging"),
        ]
        return await asyncio.gather(*tasks)

    results = asyncio.run(check_all_instances())
```

9.3 Multiprocessing — CPU-Bound Parallelism

For CPU-intensive tasks (parsing large log files, computing checksums, data transformation), multiprocessing bypasses the GIL by spawning separate processes, each with its own Python interpreter and memory space.

```
from multiprocessing import Pool

def analyze_log_file(filepath):
    error_count = 0
    with open(filepath) as f:
        for line in f:
            if "ERROR" in line:
                error_count += 1
    return (filepath, error_count)

log_files = [f"/apps/logs/pg-{i}.log" for i in range(1, 9)]

with Pool(processes=4) as pool:
    results = pool.map(analyze_log_file, log_files)
```

10. Database Access

10.1 PostgreSQL with psycopg2

```
import psycopg2
from psycopg2.extras import RealDictCursor

dsn = "host=pgprimary01 port=5432 dbname=production user=dba"

with psycopg2.connect(dsn) as conn:
    with conn.cursor(cursor_factory=RealDictCursor) as cur:
        # Database sizes
        cur.execute("""
            SELECT datname, pg_size_pretty(pg_database_size(datname))
            FROM pg_database WHERE datistemplate = false
        """)
        for row in cur:
            print(f"{row['datname']}: {row['pg_size_pretty']}")

        # Replication status
        cur.execute("""
            SELECT client_addr, state, sent_lsn, replay_lsn,
                   replay_lag
            FROM pg_stat_replication
        """)
        for replica in cur:
            print(f"  {replica['client_addr']}: {replica['state']}")
```

10.2 MySQL/MariaDB with mysql-connector

```
import mysql.connector

config = {
    "host": "mariadb01",
    "port": 3306,
    "user": "dba",
    "password": "secure_pass",
    "database": "information_schema",
}

conn = mysql.connector.connect(**config)
cursor = conn.cursor(dictionary=True)

cursor.execute("""
    SELECT table_schema,
           ROUND(SUM(data_length + index_length) / 1024 / 1024, 2)
           AS size_mb
    FROM tables
    GROUP BY table_schema
    ORDER BY size_mb DESC
""")
```

```
for row in cursor:
    print(f"{row['table_schema']}: {row['size_mb']} MB")

cursor.close()
conn.close()
```

10.3 MongoDB with PyMongo

```
from pymongo import MongoClient

client = MongoClient("mongodb://mongo01:27017,mongo02:27017/"
                     "?replicaSet=rs0")
db = client.admin

# Server status
status = db.command("serverStatus")
print(f"Connections: {status['connections']['current']}")
print(f"Uptime: {status['uptime']} seconds")

# Replica set status
rs_status = db.command("replSetGetStatus")
for member in rs_status["members"]:
    print(f"  {member['name']}: {member['stateStr']}")

# Collection operations
app_db = client.production
users = app_db.users
user_count = users.count_documents({})
recent_users = users.find({"created": {"$gte": last_week}})
```

11. Networking, HTTP, and API Development

11.1 HTTP Client with requests

```
import requests

# Query Kubernetes API
response = requests.get(
    "https://k8s-api:6443/api/v1/namespaces/database/pods",
    headers={"Authorization": f"Bearer {token}"},
    verify="/etc/kubernetes/pki/ca.crt",
    timeout=10,
)
pods = response.json()["items"]
```

11.2 Building APIs with FastAPI

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="DBA Monitoring API")

class HealthResponse(BaseModel):
    host: str
    status: str
    replication_lag: float | None

@app.get("/health/{host}", response_model=HealthResponse)
async def check_host(host: str):
    status = await get_pg_status(host)
    if not status:
        raise HTTPException(status_code=404,
                            detail=f"Host {host} not found")
    return HealthResponse(
        host=host,
        status=status["state"],
        replication_lag=status.get("lag"),
    )
```

12. Subprocess and OS-Level Automation

```
import subprocess
from pathlib import Path

def run_pg_basebackup(host, backup_dir="/apps/backups"):
    """Take a base backup from a PostgreSQL primary."""
    backup_path = Path(backup_dir) / f"base_{host}_{date_stamp()}"
    backup_path.mkdir(parents=True, exist_ok=True)

    result = subprocess.run(
        [
            "pg_basebackup",
            "-h", host,
            "-D", str(backup_path),
            "-Ft",            # tar format
            "-z",             # gzip compression
            "-P",             # progress
            "--wal-method=stream",
            "--checkpoint=fast",
        ],
        capture_output=True,
        text=True,
        timeout=3600,        # 1 hour timeout
    )

    if result.returncode != 0:
        raise RuntimeError(f"Backup failed: {result.stderr}")

    return backup_path

def run_kubectl(namespace, *args):
    """Execute a kubectl command and return parsed output."""
    cmd = ["kubectl", "-n", namespace, "-o", "json"] + list(args)
    result = subprocess.run(cmd, capture_output=True, text=True)
    result.check_returncode()
    return json.loads(result.stdout)
```

13. Kubernetes Client and Infrastructure Scripting

```
from kubernetes import client, config

# Load kubeconfig or in-cluster config
config.load_kube_config()   # from ~/.kube/config
# config.load_incluster_config()  # when running inside a pod

v1 = client.CoreV1Api()
apps_v1 = client.AppsV1Api()

# List all PostgreSQL pods
pods = v1.list_namespaced_pod(
    namespace="database",
    label_selector="app.kubernetes.io/name=percona-postgresql"
)
for pod in pods.items:
    print(f"{pod.metadata.name}: {pod.status.phase}")

# Scale a StatefulSet
apps_v1.patch_namespaced_stateful_set_scale(
    name="pg-cluster",
    namespace="database",
    body={"spec": {"replicas": 3}},
)

# Execute a command inside a pod
from kubernetes.stream import stream
resp = stream(
    v1.connect_get_namespaced_pod_exec,
    "pg-cluster-0",
    "database",
    command=["psql", "-c", "SELECT pg_is_in_recovery();"],
    container="postgres",
    stdout=True,
)
```

14. Testing and Code Quality

14.1 pytest Fundamentals

```
# test_health.py
import pytest
from dba_toolkit.pg.health import check_connection

def test_connection_success(mocker):
    mock_conn = mocker.patch("psycopg2.connect")
    mock_conn.return_value.__enter__ = lambda s: s
    mock_conn.return_value.__exit__ = lambda s, *a: None

    result = check_connection("localhost", 5432)
    assert result["status"] == "healthy"

def test_connection_timeout(mocker):
    mocker.patch("psycopg2.connect",
                 side_effect=psycopg2.OperationalError("timeout"))

    result = check_connection("localhost", 5432)
    assert result["status"] == "error"
    assert "timeout" in result["message"]

@pytest.fixture
def pg_test_db():
    """Create a temporary test database."""
    conn = create_test_db()
    yield conn
    conn.close()
    drop_test_db()
```

15. Advanced Topics

15.1 Decorators

```
import functools
import time

def retry(max_attempts=3, delay=1, backoff=2):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            attempt = 0
            while attempt < max_attempts:
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    attempt += 1
                    if attempt == max_attempts:
                        raise
                    wait = delay * (backoff ** (attempt - 1))
                    time.sleep(wait)
        return wrapper
    return decorator

@retry(max_attempts=5, delay=2)
def connect_to_primary():
    return psycopg2.connect(dsn, connect_timeout=5)
```

15.2 Generators

```
def stream_pg_log(log_path, follow=False):
    """Generator that streams PostgreSQL log lines."""
    with open(log_path) as f:
        if follow:
            f.seek(0, 2)  # go to end
        while True:
            line = f.readline()
            if line:
                yield parse_log_line(line)
            elif follow:
                time.sleep(0.1)
            else:
                return

# Usage — processes one line at a time, constant memory
for entry in stream_pg_log("/apps/logs/postgresql.log"):
    if entry.severity == "ERROR":
        send_alert(entry)
```

15.3 Context Managers

```
from contextlib import contextmanager

@contextmanager
def pg_transaction(dsn, readonly=False):
    conn = psycopg2.connect(dsn)
    try:
        if readonly:
            conn.set_session(readonly=True)
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

# Usage
with pg_transaction(dsn) as conn:
    with conn.cursor() as cur:
        cur.execute("UPDATE config SET value = %s WHERE key = %s",
                    ("256MB", "work_mem"))
```

15.4 Type Hints

```
from typing import Optional
from dataclasses import dataclass

@dataclass
class ReplicationStatus:
    client_addr: str
    state: str
    sent_lsn: str
    replay_lsn: str
    lag_seconds: float

def get_replicas(host: str,
                 port: int = 5432) -> list[ReplicationStatus]:
    """Retrieve replication status for all standbys."""
    ...

def find_lagging(replicas: list[ReplicationStatus],
                 threshold: float = 5.0
                 ) -> Optional[list[ReplicationStatus]]:
    lagging = [r for r in replicas if r.lag_seconds > threshold]
    return lagging if lagging else None
```

Part II

Go: Fundamentals to Advanced

Go (Golang) is a statically typed, compiled language created at Google in 2009 by Robert Griesemer, Rob Pike, and Ken Thompson. It was designed to solve real problems in large-scale systems engineering: slow compilation, complex dependency management, difficult concurrency, and the gap between the safety of C++ and the productivity of Python.

This part covers Go from fundamentals through advanced patterns, with all examples oriented toward the infrastructure and database automation domain.

16. Language Overview and Philosophy

Go's design philosophy is deliberate simplicity. The language has 25 keywords (Python has 35, Java over 50). There are no classes, no inheritance, no exceptions, no generics until Go 1.18 (2022), and no implicit type conversions. This minimalism is intentional — it reduces the number of ways to express the same idea, making Go code consistent and readable across large teams and codebases.

16.1 How Go Executes Code

When you run 'go build main.go', the Go compiler produces a single statically linked native binary for your target platform. This binary includes the Go runtime (goroutine scheduler, garbage collector, stack manager) but has zero external dependencies — no shared libraries, no runtime interpreter, nothing to install on the target machine.

You compile once, copy the binary to your AlmaLinux server, and it runs. No Python interpreter to install, no virtual environment to create, no pip packages to resolve. This is transformative for distributing tools across infrastructure.

16.2 Cross-Compilation

Go can compile binaries for any supported OS and architecture from any development machine:

```
# Build for Linux AMD64 from any machine
GOOS=linux GOARCH=amd64 go build -o dba-tool main.go

# Build for Linux ARM64 (e.g., Graviton instances)
GOOS=linux GOARCH=arm64 go build -o dba-tool-arm main.go

# The resulting binary has ZERO dependencies
scp dba-tool alma-server:/usr/local/bin/
ssh alma-server /usr/local/bin/dba-tool --check
```

16.3 Static Typing

Every variable has a type determined at compile time. The compiler catches type mismatches before your code ever runs, eliminating entire categories of runtime errors that are common in Python.

```
var port int = 5432         // explicit type
host := "pgprimary01"       // type inferred as string
var lag float64 = 0.045     // explicit float64

// This would not compile:
// port = "not a number"    // compile error: cannot use string as int
```

17. Environment Setup and Tooling

17.1 Installation on AlmaLinux

```
# Download and install Go
wget https://go.dev/dl/go1.22.2.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.22.2.linux-amd64.tar.gz

# Add to PATH in ~/.bashrc
export PATH=$PATH:/usr/local/go/bin
export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin

# Verify
go version
```

17.2 Project Initialization

```
# Create a new project
mkdir -p ~/projects/dba-tool && cd ~/projects/dba-tool
go mod init github.com/yourorg/dba-tool

# This creates go.mod — similar to pyproject.toml
# Dependencies are managed automatically

# Add a dependency
go get github.com/jackc/pgx/v5
go get go.mongodb.org/mongo-driver/mongo

# The go.sum file locks exact versions (like pip freeze)
```

17.3 Key Tools

- go build — Compiles to a native binary. The only build step you need.
- go run — Compiles and runs in one step. Useful during development.
- go test — Built-in testing framework. No external test runner needed.
- go fmt — Canonical code formatter. Not optional — the community treats unformatted code as a bug.
- go vet — Static analysis for common mistakes.
- golangci-lint — Aggregated linter (third-party). Runs dozens of linters in parallel.
- dlv (delve) — Go debugger. Supports breakpoints, goroutine inspection, and core dump analysis.

18. Core Syntax and Data Types

18.1 Variables and Basic Types

```
package main

import "fmt"

func main() {
    // Explicit declaration
    var port int = 5432
    var host string = "pgprimary01"
    var lagSeconds float64 = 0.045
    var isPrimary bool = true

    // Short declaration (type inferred)
    dataDir := "/apps/pgsql_data/16"
    maxConns := 200

    // Constants
    const walSegmentSize = 16 * 1024 * 1024  // 16 MB

    // Zero values — Go initializes all variables
    var count int       // 0
    var name string     // ""  (empty string)
    var ready bool      // false

    fmt.Printf("Host: %s, Port: %d\n", host, port)
}
```

18.2 Collections

Arrays and Slices

```
// Arrays — fixed size, rarely used directly
var hosts [3]string
hosts[0] = "pg01"

// Slices — dynamic, the workhorse collection
replicas := []string{"standby1", "standby2", "standby3"}
replicas = append(replicas, "standby4")

// Slice operations
firstTwo := replicas[:2]
lastOne := replicas[len(replicas)-1]

// Make with capacity hint
dbSizes := make([]int64, 0, 100)
```

Maps

```
// Maps — key-value pairs (like Python dicts)
pgConfig := map[string]string{
    "shared_buffers":      "8GB",
    "work_mem":            "256MB",
    "max_connections":     "200",
    "wal_level":           "replica",
}

// Access with comma-ok pattern
value, exists := pgConfig["work_mem"]
if exists {
    fmt.Println("work_mem:", value)
}

// Delete a key
delete(pgConfig, "wal_level")

// Iterate
for key, val := range pgConfig {
    fmt.Printf("%s = %s\n", key, val)
}
```

Structs — Go's Primary Data Structure

```
type ReplicationStatus struct {
    ClientAddr string
    State      string
    SentLSN    string
    ReplayLSN  string
    LagSeconds float64
}

// Create a struct
status := ReplicationStatus{
    ClientAddr: "10.0.1.20",
    State:      "streaming",
    LagSeconds: 0.003,
}

// Access fields
fmt.Printf("Lag: %.3fs\n", status.LagSeconds)
```

19. Control Flow and Functions

19.1 Conditionals

```
lag := getReplicationLag()

if lag > 30 {
    alert("CRITICAL: replication lag exceeds 30 seconds")
    initiateFailoverCheck()
} else if lag > 5 {
    alert("WARNING: replication lag is elevated")
} else {
    log.Printf("Replication lag normal: %.3fs", lag)
}

// if with initialization statement (unique to Go)
if conn, err := pgx.Connect(ctx, dsn); err != nil {
    log.Fatal("connection failed:", err)
} else {
    defer conn.Close(ctx)
    // use conn
}
```

19.2 Switch

```
// Go's switch is cleaner — no fallthrough by default
switch dbType {
case "postgresql":
    checkPGHealth(host)
case "mysql", "mariadb":
    checkMySQLHealth(host)
case "mongodb":
    checkMongoHealth(host)
default:
    log.Printf("Unknown database type: %s", dbType)
}

// Type switch
func describe(i interface{}) string {
    switch v := i.(type) {
    case string:
        return "string: " + v
    case int:
        return fmt.Sprintf("integer: %d", v)
    default:
        return fmt.Sprintf("unknown: %T", v)
    }
}
```

19.3 Functions

Go functions can return multiple values. This is the foundation of Go's error handling pattern.

```
// Multiple return values
func connectPG(host string, port int) (*pgx.Conn, error) {
    dsn := fmt.Sprintf("host=%s port=%d dbname=postgres", host, port)
    conn, err := pgx.Connect(context.Background(), dsn)
    if err != nil {
        return nil, fmt.Errorf("connecting to %s:%d: %w",
                               host, port, err)
    }
    return conn, nil
}

// Named return values
func getDBSize(conn *pgx.Conn, db string) (sizeBytes int64,
                                           err error) {
    err = conn.QueryRow(context.Background(),
        "SELECT pg_database_size($1)", db).Scan(&sizeBytes)
    return  // returns sizeBytes and err implicitly
}

// Variadic functions
func logEvent(severity string, parts ...string) {
    msg := strings.Join(parts, " ")
    log.Printf("[%s] %s", severity, msg)
}
```

20. Structs, Interfaces, and Methods

20.1 Methods on Structs

```
type DatabaseConnection struct {
    Host     string
    Port     int
    Database string
    conn     *pgx.Conn
}

// Method with pointer receiver (can modify the struct)
func (dc *DatabaseConnection) Connect(ctx context.Context) error {
    dsn := fmt.Sprintf("host=%s port=%d dbname=%s",
        dc.Host, dc.Port, dc.Database)
    conn, err := pgx.Connect(ctx, dsn)
    if err != nil {
        return err
    }
    dc.conn = conn
    return nil
}

func (dc *DatabaseConnection) Execute(ctx context.Context,
    query string) (pgx.Rows, error) {
    return dc.conn.Query(ctx, query)
}

func (dc *DatabaseConnection) Close(ctx context.Context) {
    if dc.conn != nil {
        dc.conn.Close(ctx)
    }
}
```

20.2 Interfaces — Implicit Implementation

Go interfaces are satisfied implicitly. A type implements an interface simply by having all the required methods — no 'implements' keyword. This is one of Go's most powerful design decisions.

```
type HealthChecker interface {
    Check(ctx context.Context) error
    Name() string
}

// PGHealthCheck implements HealthChecker implicitly
type PGHealthCheck struct {
    Host string
    Port int
}

func (h *PGHealthCheck) Check(ctx context.Context) error {
```

```
    conn, err := pgx.Connect(ctx,
        fmt.Sprintf("host=%s port=%d", h.Host, h.Port))
    if err != nil {
        return err
    }
    defer conn.Close(ctx)
    return conn.Ping(ctx)
}

func (h *PGHealthCheck) Name() string {
    return fmt.Sprintf("PostgreSQL @ %s:%d", h.Host, h.Port)
}

// Works with any HealthChecker
func runChecks(checkers []HealthChecker) {
    for _, c := range checkers {
        if err := c.Check(context.Background()); err != nil {
            log.Printf("[FAIL] %s: %v", c.Name(), err)
        } else {
            log.Printf("[ OK ] %s", c.Name())
        }
    }
}
```

20.3 Embedding — Composition Over Inheritance

```
type BaseMonitor struct {
    Host    string
    Port    int
    Timeout time.Duration
}

func (b *BaseMonitor) Address() string {
    return fmt.Sprintf("%s:%d", b.Host, b.Port)
}

type PGMonitor struct {
    BaseMonitor               // embedded — PGMonitor "inherits" Address()
    Database    string
    SlotName    string
}

// PGMonitor can call Address() directly
m := PGMonitor{
    BaseMonitor: BaseMonitor{Host: "pg01", Port: 5432},
    Database:    "production",
}
fmt.Println(m.Address()) // "pg01:5432"
```

21. Packages, Modules, and Dependency Management

21.1 Package Structure

```
dba-tool/
  go.mod
  go.sum
  main.go
  cmd/
    root.go
    health.go
    backup.go
  internal/
    pg/
      connection.go
      health.go
      backup.go
    mongo/
      connection.go
      health.go
    config/
      config.go
```

The internal/ directory is special in Go — packages under it cannot be imported by code outside the module. This enforces encapsulation at the module level. The cmd/ directory typically holds CLI command definitions when using a framework like cobra.

21.2 Go Modules

```
// go.mod
module github.com/yourorg/dba-tool

go 1.22

require (
    github.com/jackc/pgx/v5 v5.5.5
    github.com/spf13/cobra v1.8.0
    go.mongodb.org/mongo-driver v1.15.0
)
```

Go modules track exact dependency versions in go.sum. When you build, the compiler includes only the code that is actually referenced — unused dependencies are not compiled into the binary. This keeps binaries lean.

22. File I/O and Data Serialization

22.1 Reading and Writing Files

```
package main

import (
    "bufio"
    "fmt"
    "os"
    "strings"
)

// Read entire file
func readConfig(path string) (string, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return "", fmt.Errorf("reading %s: %w", path, err)
    }
    return string(data), nil
}

// Stream large files line by line
func scanPGLog(path string) error {
    file, err := os.Open(path)
    if err != nil {
        return err
    }
    defer file.Close()

    scanner := bufio.NewScanner(file)
    for scanner.Scan() {
        line := scanner.Text()
        if strings.Contains(line, "ERROR") {
            processError(line)
        }
    }
    return scanner.Err()
}
```

22.2 JSON

```
import (
    "encoding/json"
    "os"
)

type PGConfig struct {
    SharedBuffers  string `json:"shared_buffers"`
    WorkMem        string `json:"work_mem"`
    MaxConnections int    `json:"max_connections"`
```

```
    WALLevel       string `json:"wal_level"`
}

// Marshal (struct -> JSON)
config := PGConfig{
    SharedBuffers:  "8GB",
    WorkMem:        "256MB",
    MaxConnections: 200,
    WALLevel:       "replica",
}
data, err := json.MarshalIndent(config, "", "  ")

// Unmarshal (JSON -> struct)
var parsed PGConfig
err = json.Unmarshal(data, &parsed)

// Read from file
file, _ := os.Open("config.json")
defer file.Close()
decoder := json.NewDecoder(file)
err = decoder.Decode(&parsed)
```

22.3 YAML (Kubernetes Manifests)

```
import "gopkg.in/yaml.v3"

type K8sManifest struct {
    APIVersion string `yaml:"apiVersion"`
    Kind       string `yaml:"kind"`
    Metadata   struct {
        Name      string `yaml:"name"`
        Namespace string `yaml:"namespace"`
    } `yaml:"metadata"`
    Spec map[string]interface{} `yaml:"spec"`
}

// Parse a Kubernetes YAML manifest
data, _ := os.ReadFile("pg-cluster.yaml")
var manifest K8sManifest
yaml.Unmarshal(data, &manifest)
```

23. Error Handling and Logging

23.1 Error as Values

Go does not have exceptions. Functions that can fail return an error as a second value. The caller must check it explicitly. This is verbose, but it makes error handling visible and intentional.

```
conn, err := pgx.Connect(ctx, dsn)
if err != nil {
    return fmt.Errorf("connecting to primary: %w", err)
}
defer conn.Close(ctx)

rows, err := conn.Query(ctx, "SELECT datname FROM pg_database")
if err != nil {
    return fmt.Errorf("querying databases: %w", err)
}
defer rows.Close()
```

23.2 Custom Error Types

```
type ReplicationLagError struct {
    Lag       float64
    Threshold float64
    Host      string
}

func (e *ReplicationLagError) Error() string {
    return fmt.Sprintf("replication lag %.3fs exceeds threshold "+
        "%.1fs on %s", e.Lag, e.Threshold, e.Host)
}

// Check for specific error types
func handleError(err error) {
    var lagErr *ReplicationLagError
    if errors.As(err, &lagErr) {
        log.Printf("Lag alert for %s: %.3fs",
                   lagErr.Host, lagErr.Lag)
    }
}
```

23.3 Structured Logging with slog

```
import "log/slog"

// JSON structured logging (Go 1.21+)
logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{
    Level: slog.LevelInfo,
}))

logger.Info("health check completed",
```

```
        "host", "pg01",
        "status", "healthy",
        "lag_seconds", 0.003,
        "connections", 142,
    )
    // Output: {"time":"...","level":"INFO","msg":"health check completed",
    //          "host":"pg01","status":"healthy","lag_seconds":0.003, ...}
```

24. Concurrency with Goroutines and Channels

24.1 Goroutines

Goroutines are Go's lightweight concurrency primitive. They are not OS threads — they are multiplexed onto a small number of OS threads by the Go runtime scheduler. Starting a goroutine costs about 2 KB of stack space (versus ~1 MB for an OS thread), so creating thousands is practical.

```
func checkAllHosts(hosts []string) {
    var wg sync.WaitGroup

    for _, host := range hosts {
        wg.Add(1)
        go func(h string) {
            defer wg.Done()
            status := checkHealth(h)
            log.Printf("%s: %s", h, status)
        }(host)
    }

    wg.Wait()  // block until all goroutines complete
}
```

24.2 Channels

Channels are typed conduits for communication between goroutines. They enforce synchronized data transfer, eliminating most race conditions by design.

```
type HealthResult struct {
    Host   string
    Status string
    Lag    float64
    Err    error
}

func checkConcurrently(hosts []string) []HealthResult {
    results := make(chan HealthResult, len(hosts))

    for _, host := range hosts {
        go func(h string) {
            lag, err := getReplicationLag(h)
            results <- HealthResult{
                Host:   h,
                Status: statusFromError(err),
                Lag:    lag,
                Err:    err,
            }
        }(host)
    }

    var all []HealthResult
```

```
    for range hosts {
        all = append(all, <-results)
    }
    return all
}
```

24.3 Context for Cancellation and Timeouts

```
func checkWithTimeout(host string) error {
    // Cancel automatically after 10 seconds
    ctx, cancel := context.WithTimeout(
        context.Background(), 10*time.Second,
    )
    defer cancel()

    conn, err := pgx.Connect(ctx,
        fmt.Sprintf("host=%s", host))
    if err != nil {
        return err // includes timeout errors
    }
    defer conn.Close(ctx)

    return conn.Ping(ctx)
}
```

24.4 Select — Multiplexing Channels

```
func monitorWithHeartbeat(dataCh <-chan Metric,
                          done <-chan struct{}) {
    ticker := time.NewTicker(30 * time.Second)
    defer ticker.Stop()

    for {
        select {
        case metric := <-dataCh:
            processMetric(metric)
        case <-ticker.C:
            log.Println("heartbeat: monitor alive")
        case <-done:
            log.Println("shutting down monitor")
            return
        }
    }
}
```

25. Database Access

25.1 PostgreSQL with pgx

```
package main

import (
    "context"
    "fmt"
    "github.com/jackc/pgx/v5/pgxpool"
)

func main() {
    ctx := context.Background()

    // Connection pool
    pool, err := pgxpool.New(ctx,
        "host=pgprimary01 port=5432 dbname=production "+
        "user=dba pool_max_conns=10")
    if err != nil {
        log.Fatal(err)
    }
    defer pool.Close()

    // Query database sizes
    rows, err := pool.Query(ctx, `
        SELECT datname,
               pg_size_pretty(pg_database_size(datname))
        FROM pg_database
        WHERE datistemplate = false`)
    if err != nil {
        log.Fatal(err)
    }
    defer rows.Close()

    for rows.Next() {
        var name, size string
        rows.Scan(&name, &size)
        fmt.Printf("%s: %s\n", name, size)
    }
}
```

25.2 MySQL/MariaDB

```
import (
    "database/sql"
    _ "github.com/go-sql-driver/mysql"
)

dsn := "dba:password@tcp(mariadb01:3306)/information_schema"
db, err := sql.Open("mysql", dsn)
```

```
if err != nil {
    log.Fatal(err)
}
defer db.Close()

db.SetMaxOpenConns(10)
db.SetMaxIdleConns(5)

rows, err := db.Query(`
    SELECT table_schema,
           ROUND(SUM(data_length+index_length)/1024/1024, 2)
    FROM tables GROUP BY table_schema ORDER BY 2 DESC`)
```

25.3 MongoDB

```
import (
    "go.mongodb.org/mongo-driver/mongo"
    "go.mongodb.org/mongo-driver/mongo/options"
    "go.mongodb.org/mongo-driver/bson"
)

client, err := mongo.Connect(ctx, options.Client().
    ApplyURI("mongodb://mongo01:27017,mongo02:27017/"+
             "?replicaSet=rs0"))
if err != nil {
    log.Fatal(err)
}
defer client.Disconnect(ctx)

// Server status
var result bson.M
client.Database("admin").RunCommand(ctx,
    bson.D{{"serverStatus", 1}}).Decode(&result)

conns := result["connections"].(bson.M)["current"]
fmt.Printf("Current connections: %v\n", conns)
```

26. Networking, HTTP, and API Development

26.1 HTTP Client

```
import (
    "encoding/json"
    "net/http"
    "time"
)

client := &http.Client{Timeout: 10 * time.Second}

resp, err := client.Get(
    "https://k8s-api:6443/api/v1/namespaces/database/pods")
if err != nil {
    log.Fatal(err)
}
defer resp.Body.Close()

var podList struct {
    Items []struct {
        Metadata struct {
            Name string `json:"name"`
        } `json:"metadata"`
    } `json:"items"`
}
json.NewDecoder(resp.Body).Decode(&podList)
```

26.2 HTTP Server

```
import "net/http"

func main() {
    mux := http.NewServeMux()

    mux.HandleFunc("GET /health/{host}", func(w http.ResponseWriter,
                                              r *http.Request) {
        host := r.PathValue("host")
        status, err := checkPGHealth(host)
        if err != nil {
            http.Error(w, err.Error(),
                       http.StatusInternalServerError)
            return
        }
        json.NewEncoder(w).Encode(status)
    })

    log.Println("Starting DBA API on :8080")
    log.Fatal(http.ListenAndServe(":8080", mux))
}
```

27. OS-Level Automation and Exec

```
import "os/exec"

func runPgBasebackup(host, backupDir string) error {
    cmd := exec.CommandContext(ctx,
        "pg_basebackup",
        "-h", host,
        "-D", backupDir,
        "-Ft",
        "-z",
        "-P",
        "--wal-method=stream",
        "--checkpoint=fast",
    )

    cmd.Stdout = os.Stdout
    cmd.Stderr = os.Stderr

    if err := cmd.Run(); err != nil {
        return fmt.Errorf("pg_basebackup failed: %w", err)
    }
    return nil
}

func runKubectl(namespace string, args ...string) ([]byte, error) {
    fullArgs := append(
        []string{"-n", namespace, "-o", "json"}, args...)
    return exec.Command("kubectl", fullArgs...).Output()
}
```

28. Kubernetes Client-Go and Operator Patterns

```
import (
    "k8s.io/client-go/kubernetes"
    "k8s.io/client-go/tools/clientcmd"
    metav1 "k8s.io/apimachinery/pkg/apis/meta/v1"
)

func main() {
    config, err := clientcmd.BuildConfigFromFlags("",
        os.Getenv("HOME")+"/.kube/config")
    if err != nil {
        log.Fatal(err)
    }

    clientset, err := kubernetes.NewForConfig(config)
    if err != nil {
        log.Fatal(err)
    }

    // List PostgreSQL pods
    pods, err := clientset.CoreV1().Pods("database").List(
        context.Background(),
        metav1.ListOptions{
            LabelSelector: "app.kubernetes.io/name=percona-postgresql",
        },
    )

    for _, pod := range pods.Items {
        fmt.Printf("%s: %s\n", pod.Name, pod.Status.Phase)
    }

    // Watch for changes (event-driven)
    watcher, _ := clientset.CoreV1().Pods("database").Watch(
        ctx, metav1.ListOptions{})
    for event := range watcher.ResultChan() {
        pod := event.Object.(*v1.Pod)
        fmt.Printf("[%s] %s\n", event.Type, pod.Name)
    }
}
```

29. Testing and Benchmarking

29.1 Unit Tests

```
// health_test.go  (must end in _test.go)
package pg

import "testing"

func TestParseReplicationLag(t *testing.T) {
    tests := []struct {
        name     string
        input    string
        expected float64
        wantErr  bool
    }{
        {"normal lag", "00:00:00.045", 0.045, false},
        {"zero lag", "00:00:00", 0.0, false},
        {"invalid", "not-a-lag", 0, true},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := parseReplicationLag(tt.input)
            if (err != nil) != tt.wantErr {
                t.Errorf("error = %v, wantErr %v", err, tt.wantErr)
                return
            }
            if got != tt.expected {
                t.Errorf("got %f, want %f", got, tt.expected)
            }
        })
    }
}
```

29.2 Benchmarks

```
func BenchmarkParseConfig(b *testing.B) {
    data := loadTestConfig()
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        parseConfig(data)
    }
}

// Run: go test -bench=. -benchmem
// Output:
// BenchmarkParseConfig-8  1234567  987 ns/op  256 B/op  4 allocs/op
```

30. Advanced Topics

30.1 Generics (Go 1.18+)

```
// Generic function — works with any comparable type
func Contains[T comparable](slice []T, item T) bool {
    for _, v := range slice {
        if v == item {
            return true
        }
    }
    return false
}

// Usage
hosts := []string{"pg01", "pg02", "pg03"}
found := Contains(hosts, "pg02")  // true

ports := []int{5432, 3306, 27017}
found = Contains(ports, 5433)     // false

// Generic struct
type Result[T any] struct {
    Value T
    Err   error
}
```

30.2 Reflection

```
import "reflect"

func printStructFields(v interface{}) {
    val := reflect.ValueOf(v)
    typ := val.Type()

    for i := 0; i < val.NumField(); i++ {
        field := typ.Field(i)
        value := val.Field(i)
        fmt.Printf("  %s (%s) = %v\n",
            field.Name, field.Type, value.Interface())
    }
}
```

Use reflection sparingly. It bypasses compile-time type safety and is significantly slower than direct field access. It is primarily useful for serialization libraries, ORM-like tools, and debugging utilities.

30.3 Build Tags and Cross-Compilation

```
//go:build linux
// +build linux
```

```
package sysinfo

import "syscall"

func GetDiskUsage(path string) (uint64, uint64, error) {
    var stat syscall.Statfs_t
    err := syscall.Statfs(path, &stat)
    if err != nil {
        return 0, 0, err
    }
    total := stat.Blocks * uint64(stat.Bsize)
    free := stat.Bfree * uint64(stat.Bsize)
    return total, free, nil
}
```

Part III

Comparison and Contrast

31. Side-by-Side Feature Comparison

Feature	Python	Go
Typing	Dynamic; type hints optional	Static; compile-time checked
Compilation	Interpreted (CPython bytecode)	Compiled to native binary
Binary Output	No; requires interpreter + deps	Single static binary, zero deps
Startup Time	100ms–2s (imports, interpreter)	~5ms (native execution)
Memory Footprint	Higher (interpreter overhead)	Lower (compiled, no VM)
Concurrency Model	Threading (GIL), asyncio, multiprocessing	Goroutines + channels (native)
Error Handling	Exceptions (try/except)	Explicit error return values
OOP Model	Classes, inheritance, mixins	Structs, interfaces, embedding
Dependency Mgmt	pip, venv, poetry, requirements.txt	Go modules (go.mod/go.sum)
Formatting	Optional (black, ruff)	Mandatory (go fmt)
Package Ecosystem	~500K packages on PyPI	~150K modules on pkg.go.dev
Learning Curve	Gentle; readable from day one	Moderate; explicit error handling is verbose
Generics	Full support (duck typing)	Since Go 1.18 (type parameters)
REPL / Interactive	Yes (python shell, ipython)	No native REPL

32. Performance Benchmarks and Characteristics

32.1 Execution Speed

Go typically executes 10–40x faster than Python for CPU-bound operations (JSON parsing, string manipulation, mathematical computation). For I/O-bound operations — database queries, HTTP calls, file reads — the difference narrows significantly because both languages spend most of their time waiting for external systems.

32.2 Memory Usage

A minimal Go binary uses ~5–10 MB of resident memory. An equivalent Python script with imported libraries typically uses 30–80 MB. For long-running services with many concurrent connections, Go's lower per-goroutine memory (2 KB initial stack) versus Python's per-thread memory (~8 MB stack) creates a significant advantage.

32.3 Concurrency Throughput

Go can comfortably manage tens of thousands of concurrent goroutines on a single machine. Python's threading model, constrained by the GIL, typically handles hundreds of threads before performance degrades. Python's asyncio can handle thousands of concurrent I/O operations, but the programming model is more complex and all called code must be async-aware.

Scenario	Python	Go
Parse 1M JSON records	~12 seconds	~0.8 seconds
HTTP health checks (100 hosts)	~2s (asyncio) / ~5s (threading)	~0.5s (goroutines)
PostgreSQL query + process	~50ms (psycopg2)	~48ms (pgx)
Binary size (health check CLI)	~30MB (PyInstaller)	~8MB (go build)
Cold start (CronJob in K8s)	~800ms	~15ms
Concurrent connections (max)	~500 threads / 10K async	~100K goroutines

These are approximate figures based on typical infrastructure workloads. Actual performance depends on hardware, driver versions, and query complexity.

33. Ecosystem and Community

33.1 Python Ecosystem Strengths

- Database drivers: psycopg2/psycopg3, mysql-connector, pymongo, redis-py — all mature with years of production hardening.
- Web frameworks: FastAPI (async, modern), Flask (lightweight), Django (full-stack). FastAPI's Pydantic integration is particularly strong for API development.
- Automation: Ansible (Python-native), Fabric, Paramiko for SSH, Click for CLI tools.
- Data processing: pandas, polars, NumPy — unmatched for log analysis and reporting.
- Infrastructure clients: boto3 (AWS), kubernetes-client, terraform CDK.

33.2 Go Ecosystem Strengths

- Infrastructure tools: Kubernetes, Docker, Terraform, ArgoCD, Trivy, Helm, Percona Operator — the infrastructure world runs on Go.
- Database drivers: pgx (PostgreSQL, high-performance), go-sql-driver/mysql, mongo-driver — all with connection pooling built in.
- CLI frameworks: cobra (used by kubectl, Hugo, Helm), urfave/cli.
- Observability: Prometheus client, OpenTelemetry SDK, gRPC — all first-class Go libraries.
- Kubernetes development: client-go, controller-runtime, kubebuilder — essential for building operators and controllers.

33.3 Community and Learning

Python has one of the largest developer communities globally, with extensive documentation, tutorials, and Stack Overflow coverage for virtually every conceivable use case. Go's community is smaller but highly focused on systems programming and infrastructure, which means the libraries and documentation that exist are directly relevant to infrastructure engineering work.

34. When to Use Python vs Go

34.1 Choose Python When

• Rapid prototyping and scripting: Writing a quick health check script, parsing a log file, or automating a one-off task. Python's iteration speed is unmatched.

• Data analysis and reporting: Processing query results, generating CSV reports, analyzing performance metrics with pandas.

• API development with complex business logic: FastAPI provides exceptional developer productivity for REST APIs that interact with databases.

• Ansible integration: Custom modules, plugins, and filters for your Ansible automation are all Python.

• Interactive exploration: Using the Python REPL or Jupyter notebooks to investigate database state, test queries, or explore APIs interactively.

• Glue code: Connecting disparate systems — calling pg_dump, processing its output, uploading to S3, notifying Slack — Python excels at orchestrating multiple tools.

34.2 Choose Go When

• Distributing CLI tools across servers: When you need to deploy a binary to 50 AlmaLinux servers without installing any runtime dependencies.

• Building Kubernetes operators or controllers: The entire Kubernetes ecosystem is in Go. Writing operators in Go gives you access to the full client-go library and controller-runtime framework.

• High-concurrency network services: Health check daemons, metric collectors, or proxy services that need to handle thousands of concurrent connections efficiently.

• Performance-critical tools: Log parsers, data pipeline components, or any tool where execution speed and memory efficiency matter.

• Extending infrastructure tools: Writing Terraform providers, ArgoCD plugins, or custom admission webhooks for Kubernetes.

• CronJobs in Kubernetes: Where fast startup time matters because the job runs every minute and Python's interpreter startup adds meaningful overhead.

35. Hybrid Architectures — Using Both Together

The most effective infrastructure teams do not choose one language exclusively. They use each where it fits best, and the two languages integrate cleanly.

35.1 Common Hybrid Patterns

- Python scripts calling Go binaries: A Python orchestration script (or Ansible playbook) invokes a Go-compiled health checker or backup tool via subprocess. Python handles the workflow logic; Go handles the high-performance execution.
- Go services with Python data processing: A Go HTTP service collects metrics from database instances and pushes them to a message queue. A Python consumer processes the data, generates reports, and stores them.
- Go CLI tools used in Python CI/CD pipelines: Your Jenkins or GitHub Actions pipeline (often Python-scripted) invokes Go-built tools like Terraform, kubectl, or your custom DBA utilities.
- Shared configuration: Both languages read the same YAML/JSON configuration files and share a common configuration schema.

35.2 Integration Approaches

```
# Python calling a Go binary
import subprocess
import json

result = subprocess.run(
    ["./dba-tool", "health", "--output", "json", "--host", "pg01"],
    capture_output=True, text=True,
)
health_data = json.loads(result.stdout)
```

```
// Go binary designed for scripting integration
func main() {
    outputJSON := flag.Bool("json", false, "output as JSON")
    flag.Parse()

    result := checkHealth(host)

    if *outputJSON {
        json.NewEncoder(os.Stdout).Encode(result)
    } else {
        fmt.Printf("%s: %s\n", result.Host, result.Status)
    }
}
```

35.3 Recommended Architecture for DBA Tooling

For your infrastructure work, a practical architecture combines both languages:

- Go layer: Compiled CLI tools for health checks, backup verification, WAL monitoring, and Kubernetes pod inspection. Distributed as single binaries to all servers and deployed as CronJob containers in Kubernetes.
- Python layer: FastAPI dashboards for visualization, Ansible modules for configuration management, ad-hoc scripts for data analysis, and orchestration scripts that call the Go binaries for heavy lifting.
- Shared layer: JSON as the interchange format. Go tools output JSON; Python scripts consume it. YAML for configuration files read by both.

This guide provides the foundation for both languages in the context of infrastructure and database automation engineering. The key insight is not which language is better — it is which language fits the specific problem you are solving. Python gives you speed of development and ecosystem breadth. Go gives you speed of execution and deployment simplicity. Together, they cover the full spectrum of infrastructure tooling needs.